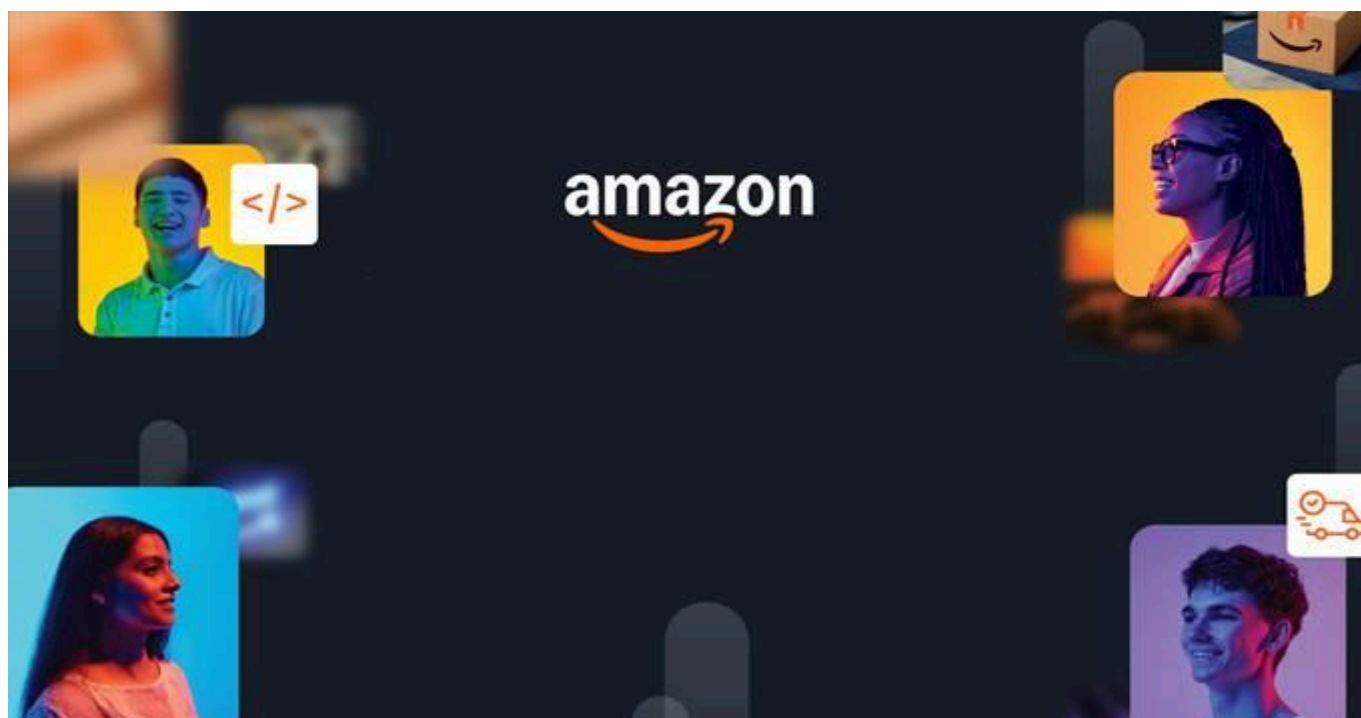




HackOn with Amazon

A Universe of Opportunity

48-Hour Hackathon | Solution Document

Team Name	<i>Bad Luck</i>
Hackathon Theme	<i>AI for Campus, Community & Everyday Life</i>
Date	<i>15 June 2026</i>

Team Members

Name	College	Role	Email
Nishant Harkut	IIIT Gwalior	Android, Cloud [AWS], Product.	College: img_2023040@iiitm.ac.in Personal : nhnishantharkut@gmail.com
Kanika Singhal	IIIT Gwalior	Backend, Frontend, UI/UX, QA	College : imt_2023038@iiitm.ac.in Personal : kanikasin216@gmail.com

1. Problem Statement & Relevance

The Problem

Students living away from home rarely lose control of money through one big purchase. It usually happens through small, repeated decisions: a late-night snack, a shared delivery cart, a local travel quote, a subscription renewal, or a skipped meal during exams because the month is already tight.

The problem is that these signals are scattered. Payment alerts live on the phone, roommate payments live in chats, food choices live in canteen menus, travel fares live in local memory, and wellness signals are usually noticed only after the student is already stressed.

Why It Matters

The affected user is a residential student on a fixed allowance, stipend, scholarship, or family transfer. This is a large and repeatable problem across campuses because millions of students live independently for the first time with limited budgets and high daily decision pressure.

The payment behavior has already moved to digital rails. In May 2026, UPI processed 23.2 billion transactions worth Rs 29.90 lakh crore. RBI-reported payments data also shows UPI carrying about 85.5% of India's payment transaction volume in the second half of CY2025. For student life, this means the spending trail already exists on the phone; the missing product layer is automated interpretation.

The student base is also large enough for this to matter beyond one campus. India's higher-education enrolment reached about 4.46 crore students in 2022-23. Even if a small fraction of residential students face monthly allowance pressure, shared-cart friction, local travel overcharging, or meal-skipping during exams, the problem is still campus-scale and repeatable.

The cost of inaction is practical:

- students run out of safe daily spending before allowance reset;
- small transactions are forgotten or double-counted;
- roommate repayments create awkward follow-ups;
- new students overpay for local travel because they do not know fair fares;
- students may delay meals during exam weeks to compensate for overspending;
- subscriptions and low-value recurring payments quietly reduce runway.

Traditional expense trackers ask students to behave like accountants. That assumption fails on campus. A busy student will not manually log every tea, bus ride, canteen meal, and shared delivery. The real need is a system that watches permitted payment signals, understands the campus context, and turns them into timely decisions.

Theme Alignment

The theme asks for AI that improves campus, community, and everyday life. PocketBuddy is built exactly around that daily campus loop:

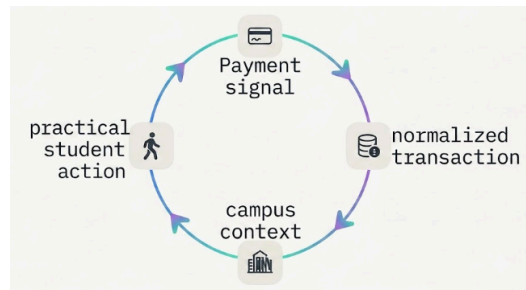
- everyday payments become budget runway;
- food gaps become meal check-ins;
- local fare reports become travel negotiation guidance;
- shared hostel purchases become verified pool flows;
- exam timing changes the tone and urgency of wellness nudges.

In our product, The AI is not presented as a generic chatbot. It is attached to concrete campus workflows where context matters.

What Makes This Novel

PocketBuddy's novelty is the automation-first loop. The product does not start with a blank expense form. It starts from the student's actual payment notifications, then layers campus context over them: hostel, mess routine, shared pools, local fare ranges, subscription habits, and exam timing.

The core loop is:



Existing products usually solve one slice: budgeting, food ordering, rides, subscriptions, or wellness. PocketBuddy connects them at the point where the student actually makes decisions. This makes the product disruptive because it turns scattered student-life signals into a campus-aware guardrail.

The practical difference is time. A student does not waste precious time maintaining a ledger. PocketBuddy turns everyday signals into actions: reduce today's spend, join a pool, eat before the food gap becomes too long, negotiate a fair ride, or verify a roommate repayment.

2. Customer & Solution

Target Customer

The primary user is a college student living in a hostel, dorm, PG, or shared room. They receive a monthly allowance or stipend, spend digitally, coordinate with roommates, and make food/travel decisions under time pressure. They need:

- low-friction tracking;
- early warning before money becomes unsafe;
- shared-expense workflows that match hostel life;
- practical guidance for food, travel, subscriptions, and wellness.

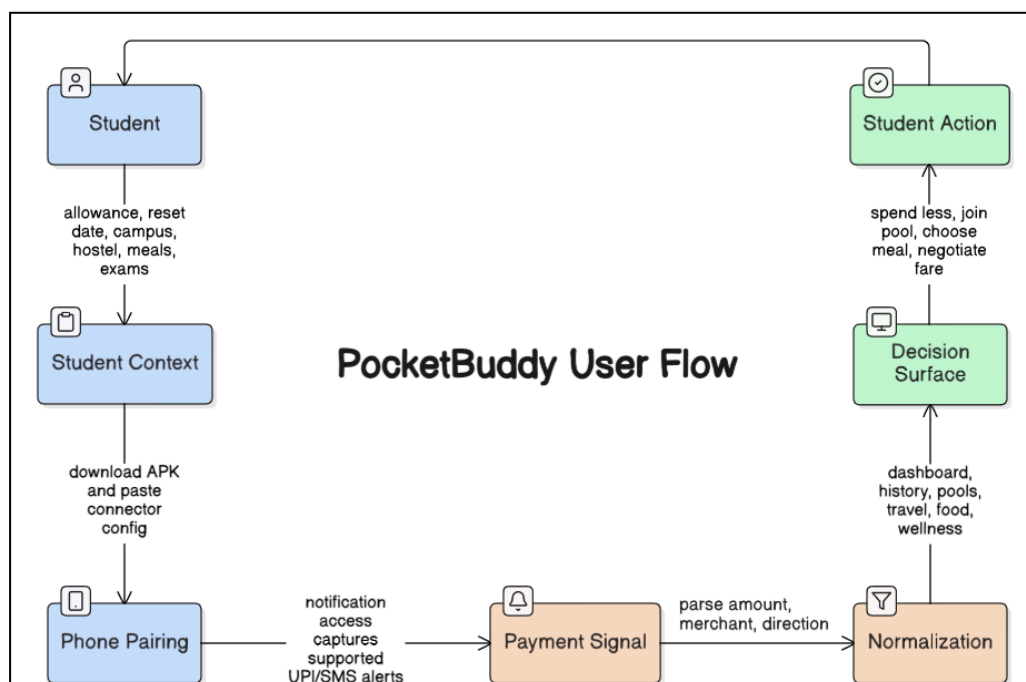
How We Solve It

PocketBuddy is a campus financial guard. It connects a student's Android phone, reads supported payment and SMS notifications after permission, and turns those events into runway, insights, and actions in the web app.

Key features:

1. **Passive Android Payment Sync** The student pairs the phone once. After that, supported UPI/SMS alerts flow into PocketBuddy without manual logging. The system parses amount, merchant, direction, and transaction reference, masks sensitive text, and deduplicates duplicate app/SMS alerts.
2. **Runway Dashboard** The dashboard answers the question students actually care about: "Can I last until reset?" It shows remaining allowance, safe daily limit, recent synced payments, category trends, and exportable history.
3. **Food & Wellness Guardrail** PocketBuddy links spending and routine. If no food transaction appears for a long stretch, especially during exam context, it can ask whether the student ate in mess, cooked, ordered, or skipped. Bedrock/Nova Lite powers contextual campus nudges where enabled.
4. **Wing Cart Pooler** Students create shared cart pools for a room or wing, add items, track progress, share the pool, and verify repayments. Incoming credit notifications can auto-verify pending UTR-based repayments when the host receives money.
5. **Travel Fare Guard** Students compare a driver quote with expected local fare ranges and receive a practical negotiation script. This protects first-year students, visitors, and anyone entering an unfamiliar campus city.

User Workflow



Working Prototype

- Demo: [\[Video/Deployed URL\]](#)
- Live deployment: <https://d3g6cg7q9hn7hi.cloudfront.net/>
- Android APK: <https://d3g6cg7q9hn7hi.cloudfront.net/downloads/PocketBuddy-Connector-v0.1.0.apk>
- Repository: <https://github.com/nishantharkut/PocketBuddy>



CAMPUS GUARD

< Collapse

+ LOG TRANSACTION

Dashboard

History

Stats

Pool

Travel

COMPANION LINKED

Last sync shown in Companion

H Nishant

harkutnishant27@gmail.com

Dark Mode

DASHBOARD

BH-2 Wing B

SPENDING UP 289% THIS WEEK

You're spending significantly more than last week. At this pace your runway shrinks by ~29 extra days.

SUBSCRIPTION BLEED WARNING

₹337/month in active subscriptions. With your current runway, consider pausing non-essential ones.

STUDENT WELLNESS INDEX

WATCH

55 / 100 WELLNESS SCORE

A few patterns need attention: your food timing, spending pace, or exam pressure is starting to stack up. Pick one reset today: a proper meal, a low-spend window, or a short break.

CONTRIBUTING SIGNALS

Avg Food gap: 8.0h

Runway: 0 days

Late-night spending: 0 txns

Spend velocity: 1.50x

Exam period: No

Social index: 0d idle

SURVIVE UNTIL BROKE

LIVE COUNTDOWN

00 h 00 m 00 s

Estimated exact date and time your allowance will run out based on your 7-day spending pace.

CAMPUS INTELLIGENCE

BEDROCK

You have spent Rs 1322 in the last 7 days, leaving you with a remaining budget of Rs 7000. Consider opting for the Tea from BH-2



CAMPUS GUARD

< Collapse

+ LOG TRANSACTION

Dashboard

History

Stats

Pool

Travel

COMPANION LINKED

Last sync shown in Companion

H Nishant

harkutnishant27@gmail.com

Dark Mode

TRANSACTION HISTORY

June 2026

DAILY

CALENDAR

MONTHLY

TOTAL

INCOME

₹7,004

EXPENSES

₹1,658

TOTAL

₹5,347

All

Food

Stationery

Travel

Subscription

Other

Skincare

Income

15 MON 06.2026

₹4 ₹441

INCOME

UPI Credit

Companion

+₹1

OTHER

Kanika Singhal

Companion

₹1

VERIFY PARSER

FOOD



CAMPUS GUARD

< Collapse

+ LOG TRANSACTION

Dashboard

History

Stats

Pool

Travel

Settings

COMPANION LINKED

Last sync shown in Companion

H Nishant

harkutnishant27@gmail.com

Dark Mode

STATS & ANALYTICS

June 2026

EXPORT CSV

INCOME

₹7,004

EXPENSES

₹1,658

NET

+₹5,347

28 TRANSACTIONS

₹151 AVG DAILY

8d STREAK Max: 8d

0% VS LAST MONTH

INCOME ₹7,004

EXPENSES ₹1,658

food 37.2%

stationery 18.7%

travel 14.6%

iptl 20.3%

other 9.2%

37% Food ₹616

20% Subscription ₹337

10% Stationery ₹310

15% Travel ₹242

PocketBuddy.

CAMPUS GUARD

< Collapse

+ LOG TRANSACTION

Dashboard

History

Stats

Pool

Travel

Settings

COMPANION LINKED

Last sync shown in Companion

H Nishant

harkutnishant27@gmail.com

Dark Mode

CART POOLS

+ START A NEW CART POOL

ACTIVE (0)

COMPLETED (4)

CANCELLED (6)

ZEPTO POOL

BH-2 Wing B

COMPLETED

Host: Nishant

MIN TARGET

₹199

CURRENT CART

₹200 (2 items)

15 JUN

ZEPTO POOL

BH-2 Wing B

COMPLETED

Host: Nishant

MIN TARGET

₹199

CURRENT CART

₹200 (2 items)

15 JUN

ZEPTO POOL

BH-2 Wing B

COMPLETED

Host: Kanika

MIN TARGET

₹199

CURRENT CART

₹300 (2 items)

15 JUN

BLINKIT POOL

BH-2 Wing B

COMPLETED

Host: Nishant

MIN TARGET

₹199

CURRENT CART

₹412 (4 items)

13 JUN

PocketBuddy.

CAMPUS GUARD

< Collapse

+ LOG TRANSACTION

Dashboard

History

Stats

Pool

Travel

Settings

COMPANION LINKED

Last sync shown in Companion

H Nishant

harkutnishant27@gmail.com

Dark Mode

CAMPUS FARE GUARD

HOW IT WORKS

Saved ₹510

PRODUCT GUIDE: DYNAMIC FARE INDEX

1 Ride App Baselines

Standard ride booking apps (Uber, Ola, Rapido) initialize the baseline fares based on simulated distance in kilometers.

2 Student Feedback loop

When you take a ride, report the actual fare paid. The system automatically recalculates the boundary range in real time.

3 Negotiation script

Use the Hindi student scripts to easily quote normal rates and avoid getting overcharged at terminal exits.

CAMPUS: ABV-IIITM GWALIOR

+ ADD ROUTE

GWALIOR RAILWAY STATION -> ABV-IIITM...

VERIFIED

TRAVEL FROM GWALIOR RAILWAY STATION TO ABV-IIITM...

ABV-IIITM Gwalior

GWALIOR AIRPORT

VERIFIED

TRAVEL FROM RAJMATA VIJAYA RAJE SCINDIA AIRPORT ...

ABV-IIITM

GWALIOR BUS STAND

VERIFIED

TRAVEL FROM GOLA KA MANDIR BUS STAND TO ABV-IIITM...

ABV-IIITM

FAIR FARE RANGES

Realistic travel rates. Seeded values dynamically updated by community reports.

Distance: 10 km

AUTO (OLA/UBER/LOCAL)

₹140 - ₹180

MIN: ₹140 MEDIAN: ₹160 MAX: ₹180

NEGOTIATION HELPER

Show this student rate script or quote it directly to local drivers.

SEEDDED CAMPUS SCRIPT

"Bhaiya, ABV-IIITM ka normal student fare Rs.150-170 hota hai. Rs.170 final?"

PocketBuddy.

CAMPUS GUARD

< Collapse

+ LOG TRANSACTION

Dashboard

History

Stats

Pool

Travel

Settings

COMPANION LINKED

Last sync shown in Companion

H Nishant

harkutnishant27@gmail.com

Dark Mode

COMPANION DEVICE

Companion linked

Google Messages

Last sync: 32m ago

UPI apps: Google Messages

This phone is linked to your account. New supported payment alerts can sync automatically.

Install PocketBuddy Connector

Download the Android connector, install it once, then paste the config from this page into the app.

1. Download

Open the APK link on the Android phone where UPI/SMS alerts arrive.

2. Install

If Android asks, allow installs from the browser for this one app.

3. Connect

Copy the Android config below, paste it in the connector, and enable notification access.

Google Play Protect can warn because this APK is installed outside the Play Store. If it blocks installation on a demo phone, open Play Store > Play Protect > Settings, temporarily disable app scanning, install PocketBuddy, then turn scanning back on.

Pair another phone or reconnect

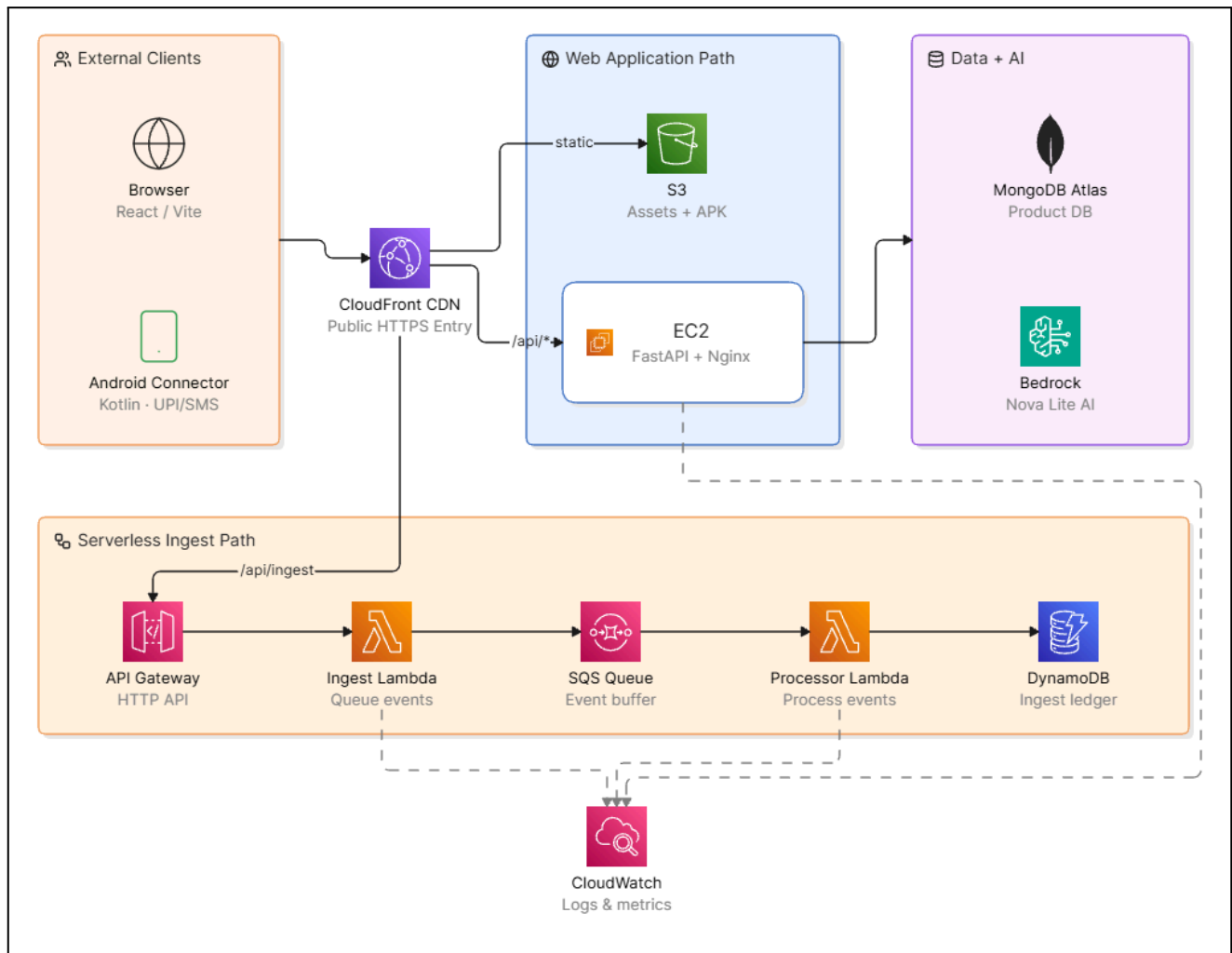
Copy this config, paste it in the Android connector, then save it on the phone.

Copy Android config

Show copied values

3. Tech Architecture & Scaling

Architecture



Architecture Highlights

- **One public entry point:** CloudFront serves the web app and routes API traffic.
- **Static delivery:** S3 stores the React build and Android APK.
- **Product API:** `/api/*` goes to Nginx on EC2, then FastAPI.
- **Primary database:** MongoDB Atlas stores users, profiles, transactions, pools, travel routes, and campus catalogs.
- **AI layer:** Bedrock Nova Lite generates travel scripts and campus nudges from selected context.
- **Separate ingest lane:** Android events use API Gateway -> Lambda -> SQS -> Lambda -> DynamoDB.
- **Why SQS:** phone notification bursts are buffered instead of hitting the backend directly.
- **Ingest ledger:** DynamoDB stores normalized mobile events for scalable, idempotent processing.
- **Observability:** CloudWatch tracks backend, Lambda, and queue logs.

Design Decision

The architecture separates user-facing product APIs from high-volume mobile ingest. The web app stays fast through CloudFront/S3, the core backend stays simple on FastAPI, and notification bursts are handled asynchronously through Lambda, SQS, and DynamoDB. This keeps the prototype practical today while leaving a clear path to scale campus-by-campus.

Tech Stack

Layer	Technology	Why
Frontend	React, Vite, TypeScript, TanStack Router, TanStack Query, Tailwind CSS	Gives a fast, typed, mobile-responsive product surface. TanStack Query is important because synced phone events can refresh dashboard/history views without turning the app into a static mockup.
Backend	Python, FastAPI, Pydantic, Motor/PyMongo	FastAPI gives strict request contracts for auth, transactions, pools, travel, food, and ingest. Motor/PyMongo keeps MongoDB operations flexible for evolving campus data.
Android	Kotlin, NotificationListenerService, OkHttp	Android notification access is the only practical way to reduce manual ledger entry. OkHttp keeps delivery simple and reliable from the companion app to the webhook.
Data/ML	MongoDB Atlas, DynamoDB, Amazon Bedrock Nova Lite	MongoDB Atlas stores user lifecycle data, profiles, transactions, pools, travel, and catalogs. DynamoDB is the append-style ledger for serverless ingest. Bedrock Nova Lite generates contextual travel and campus guidance without hosting a model.
Infra	CloudFront, S3, EC2, Nginx, API Gateway, Lambda, SQS, CloudWatch	CloudFront creates one public endpoint. S3 serves the frontend and APK. EC2/Nginx keeps the core product backend stable. API Gateway/Lambda/SQS isolate bursty phone ingest from the main backend. CloudWatch makes the demo observable.
OCR pathway	Amazon Textract integration point	Optional menu-scanning path for faster campus catalog onboarding. The product's food intelligence also works from curated campus data, so OCR is an expansion accelerator, not a core dependency.
Retrieval approach	Structured retrieval plus Bedrock context packing on MongoDB Atlas data	The current prototype retrieves scoped campus/profile data and packs it into bounded Bedrock prompts. The same Atlas data model can later support Vector Search for semantic campus memory without changing the product model.

Key Algorithms & Complexity

1. **Passive notification normalization :**

Raw payment/SMS notifications are converted into a normalized event:

{amount, currency, direction, merchant, source, transaction_reference, confidence}

The parser uses strong signals first: debit/credit words, UPI/SMS phrases, rupee amounts, UTR/reference numbers, and merchant tokens. If a field is weak, the event is still logged with lower confidence instead of being silently dropped. This is important for a campus product because students should not lose events just because one bank uses a different SMS format.

Complexity: $O(n)$ over notification text length, where n is small. In practice this behaves as constant time per phone alert.

2. **Duplicate-event suppression :**

One real payment can generate both a payment-app notification and a bank SMS. PocketBuddy avoids double-counting by using a layered identity strategy:

primary key: transaction_reference / UTR

fallback key: amount + direction + merchant + bounded time window

Complexity: $O(1)$ for fingerprint creation plus an indexed bounded-window lookup. This keeps ingest cheap even when many phones sync at the same time.

3. **Privacy-preserving sync logging :**

Notification previews are masked before storage. Long digit sequences, links, transaction references, and sensitive fragments are replaced with placeholders. This keeps logs useful without storing raw personal payment alerts.

Design choice: store enough context to explain and build trust, but not raw private bank messages.

4. **Runway and safe-spend calculation :**

The dashboard computes the student's monthly state:

remaining_allowance = monthly_allowance + income - expenses

days_left = reset_date - today

safe_daily_spend = remaining_allowance / max(days_left, 1)

Complexity: O(t) over transactions in the current cycle, with normal database indexing by user and date. The algorithm is intentionally explainable: students can trust the number because it maps directly to their allowance cycle.

5. **Food-gap and exam-context wellness state :**

PocketBuddy treats wellness as a practical state machine, not a diagnosis.

Example: if there is no food transaction for 16-17 hours during an exam window, the product can ask whether the student ate in mess, cooked, ordered, or skipped. This avoids a false assumption that "no payment" always means "no meal."

Complexity: O(1) after fetching the user's recent bounded transaction window.

6. **Cart pool payment verification :**

Pools maintain item-level contributions, participant states, UTR submissions, and host verification. Incoming credit notifications are matched against pending pool repayments using:

amount + direction = credit + payer alias/merchant text + UTR/reference + pool pending state

Complexity: O(k) over a small set of pending repayments for the host, or O(log k) with an indexed pending-payment lookup. This is a high-value campus workflow because it reduces awkward manual follow-ups after shared orders.

7. **Travel overcharge coefficient :**

Travel guidance compares a quoted fare to a route median:

overcharge_factor = quoted_fare / community_median_fare

This powers warnings, expected ranges, and negotiation copy. It is simple enough to explain to a first-year student while still capturing the core problem: uncertainty in a new city creates overpayment risk

8. **Bounded Bedrock context packing**

Bedrock/Nova Lite is used only after PocketBuddy selects the relevant context: route, quote, campus fare range, runway, food options, meal gap, and exam state. The model is not asked to inspect the whole database.

selected_context -> concise prompt -> short action message

Complexity: O(c) where c is the bounded context size. This controls cost, latency, and privacy while making the AI output grounded in actual product data.

9. **Catalog expansion path** The food catalog and travel routes use defaults for demo readiness, but they are not product limits. The long-term path is moderated campus data: users and admins add colleges, routes, canteens, payment providers, cart platforms, and category rules without changing application code.

10. **Menu OCR boundary** A photo-menu scanner was implemented as a candidate ingestion path for campus menu onboarding. The product does not depend on OCR for the final demo because campus food intelligence also works from curated/catalog data. This keeps the core user flow stable while leaving menu photo ingestion as an expansion path.

Scaling Strategy

PocketBuddy is designed to scale beyond one campus by separating static delivery, product APIs, and mobile event ingestion.

- CloudFront and S3 scale the frontend globally.
- API Gateway and Lambda accept mobile ingest events without tying them to the EC2 backend.
- SQS absorbs bursts from many Android devices and protects downstream processors.
- DynamoDB stores append-style ingest events with serverless scaling.
- FastAPI can later move from EC2 to ECS/Fargate or Lambda without changing client contracts.
- Catalogs are defaults, not hard limits: colleges, payment providers, transaction categories, travel routes, and cart platforms can be extended.

At 100x-1000x growth, the platform can add more backend instances and queue workers. At 1000x growth, ingest remains event-driven: API Gateway accepts events, SQS buffers bursts, Lambda processors normalize events independently, and DynamoDB stores the event ledger. Campus data is also configurable, so new colleges, food menus, travel routes, and cart platforms can be added without rewriting product logic.

4. Future Vision

Where This Goes

In 1-3 years, PocketBuddy can become a campus affordability layer: a privacy-first system that helps students understand money, food, travel, shared purchases, subscriptions, and routine stress before problems become urgent.

The vision is not to build another expense dashboard. The vision is to reduce the number of bad last-mile decisions students make because they lack context.

Roadmap

Horizon	Milestone	Impact
0-3 mo	Harden Android parser coverage, improve onboarding, add parser feedback review, and create reliable campus seed/admin tools.	More accurate passive tracking and smoother first-run setup.
3-6 mo	Expand food menus, travel routes, cart pool templates, subscription detection, and community reporting across multiple campuses.	More campus-specific intelligence without central manual data entry.
6-12 mo	Add moderation dashboards, privacy-preserving aggregate insights, and stronger repayment automation.	Safer multi-campus operation and more reliable shared-expense workflows.
1-3 yr	Build verified campus affordability networks for food, travel, student services, and shared purchases.	Better financial resilience for students and useful affordability signals for institutions.

Multi-Segment Expansion

PocketBuddy starts with residential students, but the same model can expand to:

- first-year students moving to new cities;
- exchange students and campus visitors;
- PG and coaching-hostel communities;
- early-career workers in shared housing;
- stipend, scholarship, and fellowship programs;
- universities that want anonymized affordability and wellness indicators.

The expansion path is campus-by-campus: seed default data, allow community reporting, moderate the catalog, and grow local intelligence over time

Revenue Model

PocketBuddy can make money without selling raw student payment data.

Revenue line	Buyer	What they pay for	Why it is realistic
Student premium	Individual students	Advanced insights, longer history, smarter subscription detection, custom alerts, exports, and multi-device sync.	Students already pay for small monthly digital services. A low-cost plan in the Rs 49-99/month range is realistic only after the free product proves savings.
Campus/hostel license	Colleges, hostels, student welfare bodies	Privacy-preserving affordability dashboards, moderated campus catalogs, route/fare guardrails, and student check-in workflows.	Institutions benefit when students have fewer money-stress and basic-routine issues during exam periods.
Verified local offers	Campus vendors and student services	Opt-in, relevant offers such as meal combos, stationery, laundry, or travel partners.	This works only if offers reduce student cost and remain clearly separated from financial advice.
Partner integrations	Fintech, banking, scholarship, and student-benefit programs	Consent-based integrations for allowance, scholarship, and student-benefit workflows.	The platform already has the student context layer; partners can reduce friction without owning the whole experience.

The first monetization path should be campus pilots plus student premium. Advertising should not be the core model because trust is the product.

Value Impact

PocketBuddy creates value in three measurable ways:

1. **Student savings:** even a small 5-8% reduction in avoidable spending on food delivery, overpaid travel, subscriptions, and duplicate/shared expenses can save a student roughly **₹300-₹600 per month** on a typical ₹6,000-₹8,000 allowance.
2. **Time saved:** students do not manually log every small transaction. If a student makes 40-60 small payments per month, passive sync saves roughly **20-40 minutes per student per month**, while also reducing missed or duplicate entries.
3. **Campus-scale impact:** at 1 lakh active students, even ₹300/month saved per student means about **₹36 crore/year** in student-side savings potential. At 10 lakh students, that becomes about **₹360 crore/year**.

Revenue can start with campus pilots and student premium plans. A realistic model is a free student app, a low-cost premium tier for advanced insights, and institution/hostel licensing for privacy-preserving campus affordability dashboards.

Links:

- GitHub: <https://github.com/nishantharkut/PocketBuddy>
 - Live app: <https://d3g6cg7q9hn7hi.cloudfront.net/>
 - Android APK: <https://d3g6cg7q9hn7hi.cloudfront.net/downloads/PocketBuddy-Connector-v0.1.0.apk>
 - Demo video: uploaded as MP4 in the submission portal.
-